CSS កម្រិតខ្ពស់



ឯកសារនេះផ្តល់ចំណេះដឹងស៊ីជម្រៅអំពីបច្ចេកទេស CSS ទំនើបៗ ដែលត្រូវបានប្រើប្រាស់យ៉ាងទូលំទូលាយក្នុងការបង្កើតគេហទំព័រប្រកបដោយភាពបត់បែន និងរចនាសម្ព័ន្ធទំនើប។ យើងនឹងស្វែងយល់ពី **Flexbox**, **CSS Grid**, និង **Media Query** ដែលជាស្នូលនៃ layout design សម័យទំនើប។

មាតិកា

ឯកសារនេះត្រូវបានរៀបចំជាបីជំពូកធំៗ ដែលគ្របដណ្តប់លើបច្ចេកទេស CSS layout សំខាន់ៗ បីយ៉ាង។ រៀងរាល់ជំពូកមានការពន្យល់ ឧទាហរណ៍ជាក់ស្តែង និងតារាងយោងសម្រាប់ developer ។

1

Flexbox

CSS Layout Module សម្រាប់រៀបចំ elements ជា row ឬ column ក្នុង container មួយ

2

CSS Grid

2-Dimensional Layout System សម្រាប់គ្រប់គ្រង layout ជា rows និង columns ក្នុងពេលតែមួយ

3

Media Query

បច្ចេកទេសសម្រាប់ធ្វើ responsive design ដែលអាចប្តូរ style តាម screen size

ជំពូកទី ១

Flexbox

Flexbox គឺជាអ្វី?

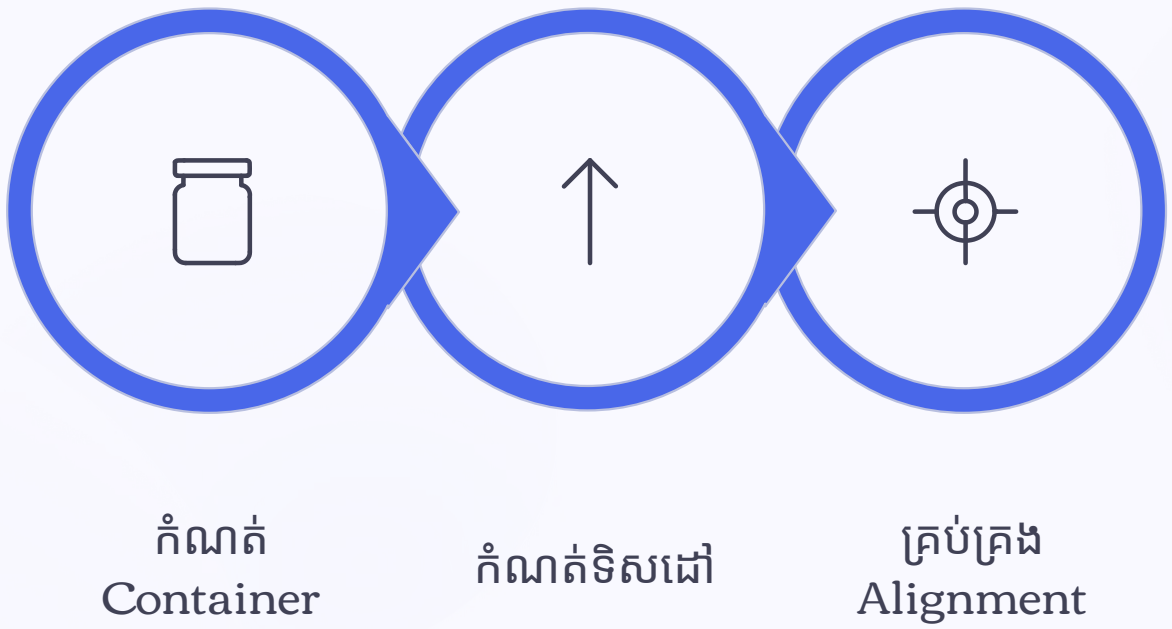
Flexbox គឺជា **CSS Layout Module** ដែលប្រើសម្រាប់រៀបចំ elements នៅក្នុង container មួយឲ្យមាន **flexible និង responsive**។ វាអាចរៀបចំ **row ឬ column**, កំណត់ **alignment**, និងចែក **space** រវាង elements បានយ៉ាងងាយស្រួល។

Flexbox ត្រូវបានបង្កើតឡើងដើម្បីដោះស្រាយបញ្ហា layout ជាច្រើន ដែល CSS ប្រពៃណីមានការលំបាក ជាពិសេសសម្រាប់ **centering** និង **distributing space**។

លក្ខណៈពិសេស

- រៀបចំ elements ជា **row ឬ column**
- គ្រប់គ្រង **alignment** ទាំងផ្តេក និងទទឹង
- ចែក **space** រវាង elements ដោយស្វ័យប្រវត្តិ
- ធ្វើ **responsive layout** បានងាយ
- ប្រើប្រាស់យ៉ាង **ទូលំទូលាយ** ក្នុង modern web design

Flexbox ត្រូវបានប្រើយ៉ាងទូលំទូលាយក្នុង **modern web design** ដើម្បីបង្កើត layout ដែលអាចប្តូរតាម screen size។ វាជាជំនួសសំខាន់សម្រាប់ float-based layouts ចាស់ៗ ហើយផ្តល់ភាពងាយស្រួល និងអានបាន code បានកាន់តែច្បាស់ ពិសេសសម្រាប់ navigation bars, card layouts, និង centering content។



Flex Container Properties

Flex Container Properties ជា properties ដែលដាក់នៅលើ **parent element** (container) ដែល define ជាអ្នកគ្រប់គ្រង layout របស់ children elements (flex items) ទាំងអស់ក្នុងវា។ Properties ទាំងនេះគ្រប់គ្រងទិសដៅ ការតម្រឹម និងការចែករំលែក space។

Property	Values	ការពណ៌នា
`display`	`flex` | `inline-flex`	ប្រកាស container ជា flex container ។ `flex` ធ្វើ block-level, `inline-flex` ធ្វើ inline-level
`flex-direction`	`row` | `row-reverse` | `column` | `column-reverse`	កំណត់ទិសដៅ main axis សម្រាប់ flex items ។ `row` ជាលំនាំដើម ដែលរៀបជាផ្តេក
`flex-wrap`	`nowrap` | `wrap` | `wrap-reverse`	គ្រប់គ្រងថាតើ flex items អាច wrap ទៅបន្ទាត់ថ្មីបានឬអត់ ពេល container ចង្អៀត
`flex-flow`	`flex-direction flex-wrap`	Shorthand សម្រាប់ `flex-direction` និង `flex-wrap` ក្នុង declaration តែមួយ
`justify-content`	`flex-start` | `flex-end` | `center` | `space-between` | `space-around` | `space-evenly`	គ្រប់គ្រងការតម្រឹម flex items តាម **main axis** (ផ្តេក ឬ ទទឹង)
`align-items`	`stretch` | `flex-start` | `flex-end` | `center` | `baseline`	គ្រប់គ្រងការតម្រឹម flex items តាម **cross axis** (ផ្ទុយពី main axis)
`align-content`	`flex-start` | `flex-end` | `center` | `space-between` | `space-around` | `stretch`	គ្រប់គ្រង space រវាង flex lines ពេល items ត្រូវ wrap ច្រើនជាង ១ បន្ទាត់
`gap`	`row-gap column-gap`	កំណត់ចន្លោះ (gap) រវាង flex items ។ ជំនួស `margin` ឲ្យមានភាពងាយស្រួល

> 💡 **គន្លឹះ:** `justify-content` ដំណើរការតាម **main axis** ខណៈ `align-items` ដំណើរការតាម **cross axis**។ ការយល់ដឹងពីភាពខុសគ្នានេះ ជាគ្រឹះសំខាន់ក្នុងការប្រើ Flexbox។

Flex Item Properties

Flex Item Properties ជា properties ដែលដាក់ដោយផ្ទាល់នៅលើ **child elements** (flex items) ក្នុង flex container។ Properties ទាំងនេះអនុញ្ញាតឲ្យ items ម្នាក់ៗអាចគ្រប់គ្រងខ្លួនឯងបានក្នុងបរិបទ flex layout ដូចជា ភាពធំ, ការដែន, និង ការតម្រឹម។

Property	Values	ការពណ៌នា
order	0 (default) | ចំនួនគត់	កំណត់លំដាប់ប្រភេទ display របស់ flex item ។ items ដែលមាន order តូចជាង នឹងបង្ហាញមុន
flex-grow	0 (default) | ចំនួនវិជ្ជមាន	កំណត់ proportion ដែល item អាចធំឡើង (grow) ដើម្បីបំពេញ space ដែលនៅសល់ក្នុង container
flex-shrink	1 (default) | ចំនួនវិជ្ជមាន	កំណត់ proportion ដែល item អាចតូចចុះ (shrink) ពេល container ចង្អៀតជាង items រួមគ្នា
flex-basis	auto | ទំហំ CSS	កំណត់ initial size របស់ flex item មុន free space ត្រូវបានចែក ។ auto ប្រើ width/height
flex	flex-grow flex-shrink flex-basis	Shorthand សម្រាប់ grow, shrink, basis ។ ផ្តល់ advice ឲ្យ ប្រើ shorthand នេះជំនួស individual
align-self	auto | flex-start | flex-end | center | stretch | baseline	Override align-items សម្រាប់ item ជាក់លាក់ ។ អនុញ្ញាតការ តម្រឹម cross axis ផ្ទាល់ខ្លួន

flex shorthand ដ៏ល្អ

ការប្រើ flex: 1 គឺស្មើនឹង flex-grow: 1; flex-shrink: 1; flex-basis: 0%។ ជួយឲ្យ items ចែក space ស្មើៗគ្នា។

align-self ពិសេស

ក្នុងករណីត្រូវការ item ណាមួយ មាន alignment ខុសពីអ្នកដទៃ ប្រើ align-self ជំនួស ។ Property នេះ override align-items របស់ container

ជំពូកទី ២

CSS Grid

CSS Grid គឺជា **CSS Layout System** ដែលប្រើសម្រាប់រៀបចំ elements នៅក្នុង **row និង column** ក្នុងពេលតែមួយ។ វាជា **2-Dimensional Layout** ដែលអាចគ្រប់គ្រង layout បានលម្អិត និងមានប្រសិទ្ធភាពសម្រាប់ការបង្កើត web page ឬ complex layout ។

ខុសពី Flexbox ដែលជា one-dimensional, Grid អនុញ្ញាតឲ្យ developer គ្រប់គ្រង layout ក្នុងទំហំ ២ dimension ក្នុងពេលតែមួយ ជួយបង្កើត page layout ស្មុគ្រស្មាញបាន យ៉ាងសាមញ្ញ ។



2D Layout System

Grid គ្រប់គ្រងទាំង **rows** និង **columns** ក្នុងពេលតែមួយ ខុសពី Flexbox ដែលគ្រប់គ្រងតែ axis តែមួយ



ទីតាំងច្បាស់លាស់

អាចកំណត់ **ទីតាំង** និង **ទំហំ** របស់ element ច្បាស់លាស់ ដោយប្រើ grid lines ឬ named areas



Responsive Design

ងាយស្រួលបង្កើត **responsive layout** ដែលប្រែប្រួលតាម screen size ដោយប្រើ `auto-fit` / `auto-fill`

- 🆚 **Flexbox vs Grid:** ប្រើ **Flexbox** សម្រាប់ layout ១ dimension (component-level) ហើយប្រើ **Grid** សម្រាប់ layout ២ dimension (page-level)។ ជាញឹកញាប់ developer ប្រើទាំងពីរក្នុង project តែមួយ។

Grid Container Properties

Grid Container Properties ជា properties ដែលកំណត់ **structure** ទូទៅ និង **behavior** នៃ grid ។ ដូចជា Flexbox, properties ទាំងនេះដាក់លើ **parent element** ហើយ define grid layout ដែល children ត្រូវ follow ។

Property	Values / Example	ការពណ៌នា
`display`	`grid` | `inline-grid`	ប្រកាស element ជា grid container ។ `inline-grid` ធ្វើ container ជា inline-level element
`grid-template-columns`	`200px 1fr 2fr` | `repeat(3, 1fr)`	កំណត់ column tracks ។ អាចប្រើ pixel, %, fr, auto, minmax(), repeat()
`grid-template-rows`	`100px auto 200px`	កំណត់ row tracks ។ ដូចគ្នា columns ប៉ុន្តែសម្រាប់ rows
`grid-template-areas`	`"header header" "sidebar main" "footer footer"`	Define named areas ក្នុង grid ។ ធ្វើ layout ឲ្យអ្នកអានបានច្បាស់ (visual)
`grid-template`	Shorthand	Shorthand សម្រាប់ rows, columns, areas ក្នុង declaration តែមួយ
`column-gap` / `row-gap`	`20px` | `1rem`	កំណត់ spacing រវាង columns ឬ rows ។ `gap` ជា shorthand ទាំងពីរ
`justify-items`	`start` | `end` | `center` | `stretch`	តម្រឹម grid items តាម **inline (row) axis** ក្នុង cell ។ ប៉ះពាល់ items ទាំងអស់
`align-items`	`start` | `end` | `center` | `stretch`	តម្រឹម grid items តាម **block (column) axis** ក្នុង cell ។ ប៉ះពាល់ items ទាំងអស់
`place-items`	`center center`	Shorthand សម្រាប់ `align-items` + `justify-items`
`justify-content`	`start` | `end` | `center` | `space-between`	តម្រឹម **grid ទាំងមូល** ក្នុង container តាម inline axis ពេល grid តូចជាង container
`align-content`	`start` | `end` | `center` | `space-around`	តម្រឹម **grid ទាំងមូល** ក្នុង container តាម block axis ពេល grid តូចជាង container
`grid-auto-flow`	`row` | `column` | `dense`	គ្រប់គ្រងរបៀប auto-placed items ត្រូវបានដាក់ (`dense` បំពេញ gaps)

ក

CSS Grid Tracks (Rows and Columns)

Grid Tracks ជាបន្ទាត់ (rows) ឬ ជួរ (columns) ដែលបង្កើតឡើងក្នុង CSS Grid ។ ពួកវាជា "ផ្លូវ" ដែល grid items ស្ថិតនៅ ហើយត្រូវបានកំណត់ដោយ `grid-template-rows` និង `grid-template-columns`។

Grid Columns

ប្រើ `grid-template-columns` ដើម្បីកំណត់ column tracks

```
/* 3 columns fixed */
grid-template-columns: 200px 200px 200px;

/* Mixed units */
grid-template-columns: 200px 1fr auto;

/* repeat shorthand */
grid-template-columns: repeat(3, 1fr);

/* minmax */
grid-template-columns: repeat(3, minmax(100px, 1fr));
```

Grid Rows

ប្រើ `grid-template-rows` ដើម្បីកំណត់ row tracks

```
/* fixed rows */
grid-template-rows: 100px 200px;

/* auto-sized rows */
grid-template-rows: auto auto;

/* repeat */
grid-template-rows: repeat(3, 100px);

/* mixed */
grid-template-rows: 80px 1fr 60px;
```

Grid tracks អាចប្រើ units ជាច្រើនប្រភេទ ដូចជា `px`, `%`, `fr` (fraction unit), `auto`, `min-content`, `max-content`, និង `minmax()`។ ការយោគយល់ units ទាំងនេះ ជួយបង្កើត grid layout ដ៏មានប្រសិទ្ធភាព។

px / rem

Fixed size ។ ទំហំ track មិនប្រែប្រួលតាម container ។ ល្អសម្រាប់ elements ដែលមានទំហំស្ថិរ

fr

Fraction unit ។ ចែករំលែក space ដែលនៅសល់ ។ `1fr 2fr` ចែក 1/3 និង 2/3

auto

ទំហំដោយស្វ័យប្រវត្តិ ។ ធំតាម content ប្រសិនបើ min/max មិនត្រូវបាន set

minmax()

Function កំណត់ minimum និង maximum size ។ ប្រើ `minmax(100px, 1fr)`

ខ

Grid Gap

Grid Gap ជា spacing (ចន្លោះ) រវាង rows និង columns ក្នុង grid ។ ក្នុង CSS modern, properties ទាំងនេះត្រូវបាន rename ពី `grid-gap` ទៅ `gap` ។ Gap ផ្តល់ spacing ស្អាត រវាង items ដោយមិនប្រើ margin ។

Properties នៃ Gap

Property	ការពណ៌នា
`row-gap`	ចន្លោះរវាង rows
`column-gap`	ចន្លោះរវាង columns
`gap`	Shorthand ទាំង row-gap + column-gap

ឧទាហរណ៍ Code

```
.grid-container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);

  /* row gap តែមួយ */
  row-gap: 20px;

  /* column gap តែមួយ */
  column-gap: 16px;

  /* shorthand: row col */
  gap: 20px 16px;

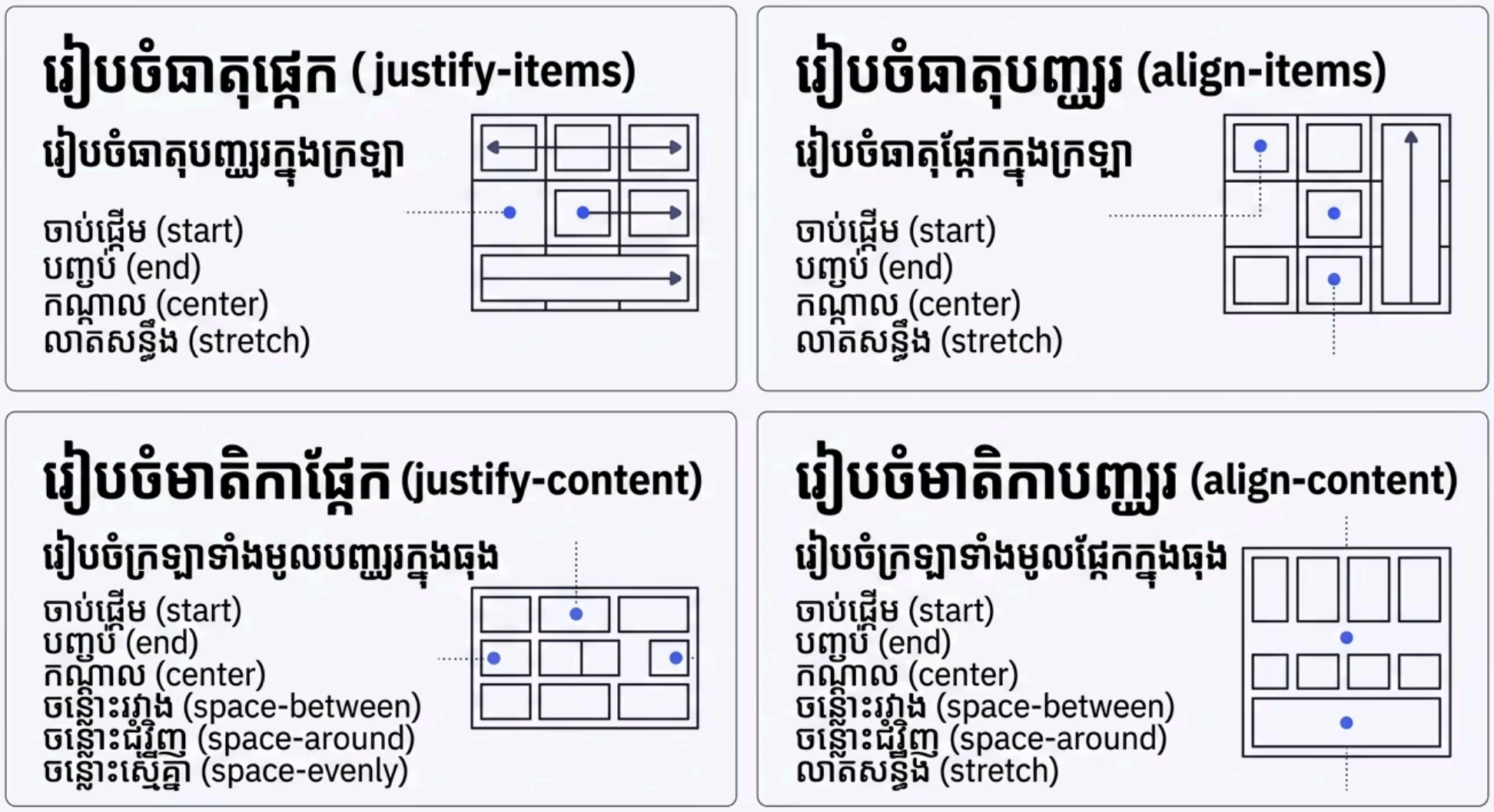
  /* shorthand ស្មើគ្នា */
  gap: 20px;
}
```

📌 **ចំណាំ:** `gap` ដំណើរការក្នុង **Grid** និង **Flexbox** ទាំងពីរ ។ Property `grid-gap` ចាស់នៅ supported ប៉ុន្តែ ផ្តល់ advice ឲ្យប្រើ `gap` ថ្មី ព្រោះ modern browsers គ្រប់ version គ្រប់គ្នា support ។

គ

Grid Align

CSS Grid ផ្តល់ properties ជាច្រើនសម្រាប់ **alignment** ។ ត្រូវយល់ភាពខុសគ្នារវាង ការតម្រឹម **items ក្នុង cells** ហើយ ការតម្រឹម **grid ទាំងមូល** ក្នុង container ។



Aligning Items (ក្នុង Cell)

```
.grid {
  /* ផ្តេក (inline axis) */
  justify-items: center;

  /* បញ្ឈរ (block axis) */
  align-items: center;

  /* shorthand */
  place-items: center center;
}
```

Aligning Grid (ក្នុង Container)

```
.grid {
  /* Grid ផ្ទាល់ - ផ្តេក */
  justify-content: space-between;

  /* Grid ផ្ទាល់ - បញ្ឈរ */
  align-content: start;

  /* shorthand */
  place-content: start space-between;
}
```

ឃ

មុខងារ repeat()

មុខងារ repeat() ជួយ developer **កាត់បន្ថយ code ដដែលៗ** ក្នុងការ define grid tracks ។ ជំនួសពី1fr 1fr 1fr 1fr, អ្នកអាចសរសេរ repeat(4, 1fr) ដែល reads ធ្ងន់ ហើយ maintain កាន់តែងាយស្រួល ។

- ### repeat() ជាមួយ fixed count

  repeat(3, 200px) – បង្កើត 3 columns, ម្នាក់ 200px

  ```
  grid-template-columns: repeat(3, 200px);
  ```

- ### repeat() ជាមួយ fr unit

  repeat(4, 1fr) – 4 columns ស្មើៗ ចែក space ដែលនៅ

  ```
  grid-template-columns: repeat(4, 1fr);
  ```

- ### auto-fill

  បំពេញ columns ឲ្យបានច្រើនតាម container size ។ Columns ទទេ ក៏ exist ដែរ

  ```
  grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));
  ```

- ### auto-fit

  ដូច auto-fill ប៉ុន្តែ collapse columns ទទេ ជំនួសឲ្យ keep them ។ Items ក្នុង row ពង្រីក

  ```
  grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
  ```

> 🔍 auto-fill **vs** auto-fit**:** ការប្រើ auto-fit ជាមួយ minmax() ជា pattern ដ៏ល្បីល្បាញ (ហៅថា **RAM pattern** - Repeat, Auto, Minmax) ដែលបង្កើត fully responsive grid ដោយ **មិនចាំបាច់ media query**!

ង

ខ្នាត FR (Fraction Unit)

Unit `fr` ជា unit ពិសេសសម្រាប់ CSS Grid ។ វាតំណាងឲ្យ **fraction** (ផ្នែក) នៃ **free space** ដែលនៅសល់ក្នុង grid container ។ ខុសពី `%`, unit `fr` គ្រប់គ្រង space **ក្រោយ** fixed-size items ត្រូវបានបែងចែក ជំនួស space ទាំងអស់ ។

របៀបដែល FR ដំណើរការ

Browser គណនា free space ដោយ ដក fixed tracks ចេញ ហើយ ចែក space ដែលនៅ ទៅ fr tracks តាម proportion ។

```
/* Container: 900px wide */
/* gap: 20px x2 = 40px */
/* Free space: 900 - 200 - 40 = 660px */
/* 1fr = 220px, 2fr = 440px */

grid-template-columns: 200px 1fr 2fr;
```

ឧទាហរណ៍ FR ជាច្រើន

```
/* ស្មើៗ 3 columns */
grid-template-columns: 1fr 1fr 1fr;

/* sidebar + main */
grid-template-columns: 1fr 3fr;

/* sidebar + main + sidebar */
grid-template-columns: 1fr 4fr 1fr;

/* ចំណូល fixed, ដែនទានដោយ fr */
grid-template-columns: 250px 1fr;

/* minmax ជាមួយ fr */
grid-template-columns: repeat(3, minmax(100px, 1fr));
```

1fr

ទំហំស្មើ

`repeat(3, 1fr)` ចែក space ស្មើៗ ៣ phần

2fr

ទ្វេដង

`1fr 2fr` – column ទីពីរ ធំជាងទីមួយ ២ ដង

fr

Grid-only

FR unit ប្រើបានតែ Grid tracks ប៉ុណ្ណោះ មិន support ក្នុង Flexbox

Grid Items Properties

Grid Item Properties ជា properties ដែលដាក់ដោយផ្ទាល់លើ **child elements** ក្នុង grid container ។ Properties ទាំងនេះអនុញ្ញាតឲ្យ item ម្នាក់ៗ កំណត់ **ទំហំ, ទីតាំង**, និង **alignment** ផ្ទាល់ខ្លួន ។

Property	Values / Example	ការពណ៌នា
grid-column-start	1 | span 2 | auto	កំណត់ column line ដែល item ចាប់ផ្តើម ។ Line ១ ជា line ខាងឆ្វេងបំផុត
grid-column-end	3 | span 2 | -1	កំណត់ column line ដែល item បញ្ចប់ ។ -1 ជា line ខាងស្តាំបំផុត
grid-column	1 / 3 | 1 / span 2	Shorthand សម្រាប់ grid-column-start / grid-column-end
grid-row-start	1 | span 2 | auto	កំណត់ row line ដែល item ចាប់ផ្តើម
grid-row-end	3 | span 3	កំណត់ row line ដែល item បញ្ចប់
grid-row	1 / 3 | 2 / span 2	Shorthand សម្រាប់ grid-row-start / grid-row-end
grid-area	header | 1 / 1 / 3 / 3	ចាត់ item ទៅ named area ឬ កំណត់ row-start/col-start/row-end/col-end
justify-self	start | end | center | stretch	Override justify-items សម្រាប់ item ជាក់លាក់ (ផ្តេក)
align-self	start | end | center | stretch	Override align-items សម្រាប់ item ជាក់លាក់ (បញ្ឈរ)
place-self	center | start end	Shorthand សម្រាប់ align-self + justify-self

ក

Naming Grid Items

CSS Grid មានមុខងារ **Named Areas** ដ៏ទាក់ទាញ ដែលអនុញ្ញាតឲ្យ developer define layout ជា visual map ក្នុង CSS ។ ការ name grid items ជួយធ្វើ code ឲ្យ **អានបានងាយ** និង **maintain បានងាយ** ។

Container: `grid-template-areas`

```
.container {
  display: grid;
  grid-template-columns: 1fr 3fr;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  gap: 20px;
  min-height: 100vh;
}
```

Items: `grid-area`

```
.header {
  grid-area: header;
}

.sidebar {
  grid-area: sidebar;
}

.main {
  grid-area: main;
}

.footer {
  grid-area: footer;
}
```

ក្នុង `grid-template-areas` អ្នកប្រើ **quotes ("")** ដើម្បី represent ១ row ។ ក្នុង row ម្នាក់ ចែករំលែក names ដោយ space ។ ប្រើ `.` (dot) ដើម្បី represent cell ទទេ ។ ឧទាហរណ៍: `"sidebar . main"` – middle column ទទេ ។

→ ប្រើ dot (.) សម្រាប់ empty cell

ក្នុង template ប្រើ `.` ដើម្បីបង្ហាញ cell ដែលគ្មាន item ។ ឧ. `"header . sidebar"`

→ Name ត្រូវ form shape ត្រឹមត្រូវ

Grid areas ត្រូវ form **rectangle** ប្រសិនបើ name ជ្រើសរើស L-shape ឬ shape ចម្លែក browser នឹង ignore

→ Responsive ដោយ redefine areas

ក្នុង media queries, redefine `grid-template-areas` ដើម្បី rearrange layout ដោយ code តិច

ខ

Grid Items - Alignment

ខណៈ container properties ដូចជា `justify-items` និង `align-items` ប៉ះពាល់ **items ទាំងអស់**, property លើ item ផ្ទាល់ (self) អនុញ្ញាតឲ្យ **override** behavior សម្រាប់ item ជាក់លាក់ ។ ភាពបត់បែននេះ ពិតជាមានប្រយោជន៍ សម្រាប់ design ស្មុគ្រស្មាញ ។

`justify-self`

តម្រឹម item ផ្តេក (inline axis) ក្នុង grid cell ។ Values: `start`, `end`, `center`, `stretch`

```
.item-special {
  justify-self: center;
}
```

`align-self`

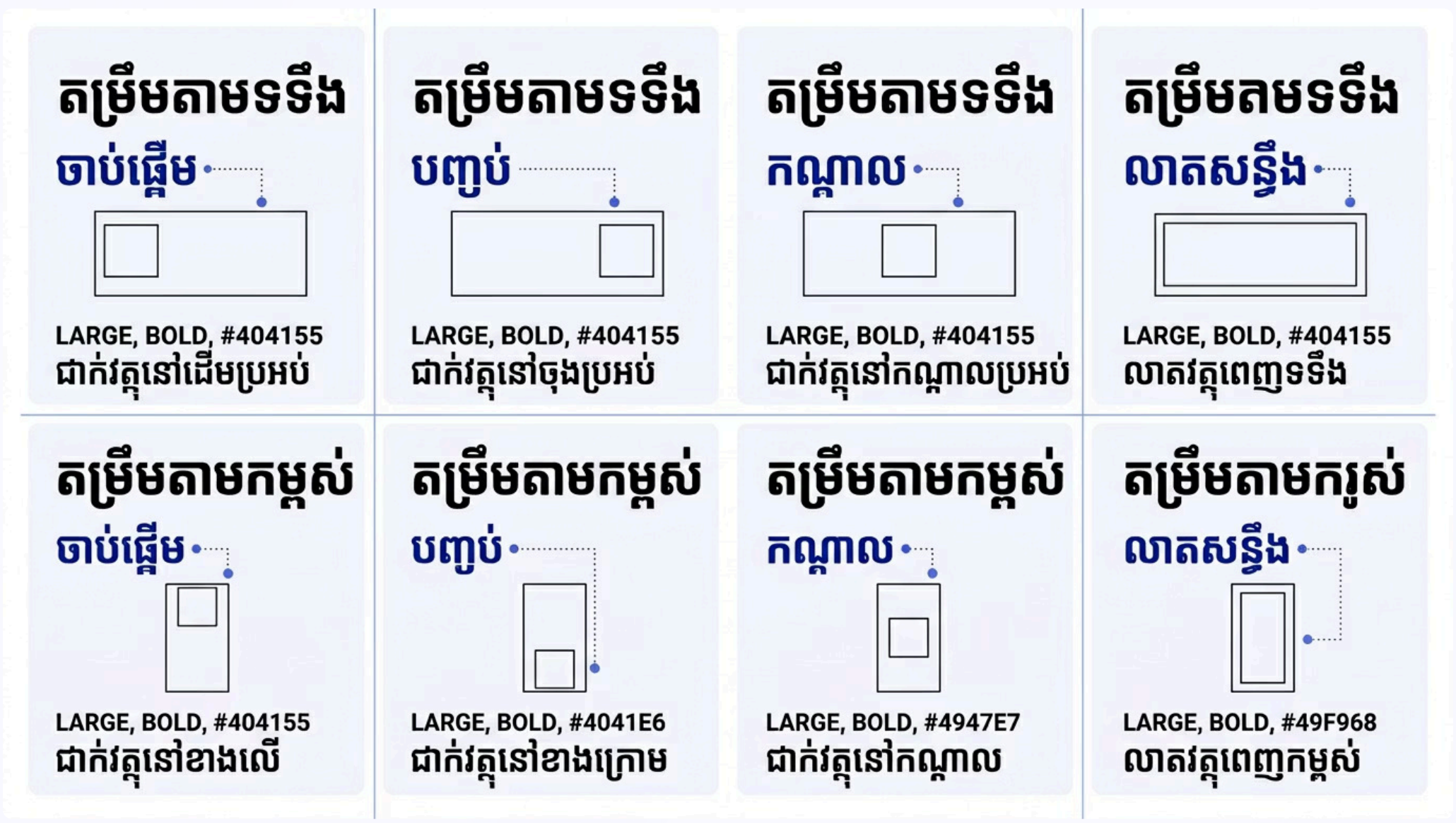
តម្រឹម item បញ្ឈរ (block axis) ក្នុង grid cell ។ Values: `start`, `end`, `center`, `stretch`

```
.item-special {
  align-self: end;
}
```

`place-self`

Shorthand ជំនួស `align-self` + `justify-self` ។ Value ទីមួយ ជា align, ទីពីរ ជា justify

```
.item-special {
  place-self: center end;
}
```



💡 **គន្លឹះ:** ប្រើ `place-self: center` (value តែមួយ apply ទាំង axes ពីរ) ជា shortcut ដ៏ងាយ ដើម្បី center item ក្នុង grid cell ។

គ

Grid Items - Order

Property `order` ក្នុង CSS Grid (ដូចជា Flexbox) អនុញ្ញាតឲ្យ developer ប្តូរ **លំដាប់ visual** របស់ grid items ដោយ **មិនប្តូរ HTML structure** ។ ដោយ default, items ត្រូវបាន render តាម DOM order (`order: 0`) ។

ឧទាហរណ៍ Code

```
/* Default order ជា 0 */
/* Items ដែល order ទាប → បង្ហាញមុន */

.item-1 { order: 3; } /* បង្ហាញចុងក្រោយ */
.item-2 { order: 1; } /* បង្ហាញដំបូង */
.item-3 { order: 2; } /* បង្ហាញទីពីរ */

/* ប្តូរ order ក្នុង media query */
@media (max-width: 768px) {
  .sidebar { order: 2; }
  .main    { order: 1; }
}
```

ចំណុចសំខាន់ Contact Order

- Default `order` ជា **0** សម្រាប់ items ទាំងអស់
- Items ដែលមាន `order` **ទាប** → បង្ហាញ **មុន**
- អាចប្រើ **ចំនួនអវិជ្ជមាន** (negative) ដើម្បីដាក់ item នៅ **ដំបូង**
- ប៉ះពាល់ **visual order** ប៉ុណ្ណោះ – **tab order** និង accessibility ស្ថិតតាម DOM
- ល្អសម្រាប់ **responsive** ដែលត្រូវ reorder content ស្ថានការណ៍ screen ខ្នាតផ្សេង

⚠️ **Accessibility Warning:** ការប្រើ `order` ដើម្បីប្តូរ visual order យ៉ាងខ្លាំង អាចបង្កើត **ការភ្ញាក់ភ្លើម** សម្រាប់ screen reader users ដែល navigate ដោយ keyboard ។ ចំពោះ reordering ធំ ពិចារណា ប្តូរ HTML structure ជំនួស ។

កំណត់ទីតាំង

ប្រើ grid-column និង grid-row ដើម្បីដាក់ទីតាំង

រួមចំណាំ និងលំដាប់

ប្រើ align-self, justify-self និង order សម្រាប់តម្រៀប



ដាក់ឈ្មោះតំបន់

ប្រើ grid-area ដើម្បីផ្តល់ឈ្មោះសម្រាប់តំបន់